Lecture 5 — Regional DP Patterns

> Regional Level · Prerequisites: Year Curriculum Weeks 7 (prefix sums), 12 (monotonic queue), 14–15 (DP I and II) · Estimated time: 2.5 hours

1. Overview

By now you can design DP states. At regional level, the typical DP problem has a twist on top of a correct state:

- the obvious transition is **too slow** by a factor of n, but it sums or maximises over a contiguous range of previous states, so prefix sums or a monotonic deque remove the factor;
- the question asks to **count subsequences** (occurrences of a pattern, distinct subsequences, subsequences over all wildcard fillings);
- the answer must be **reconstructed**, often the lexicographically smallest one.

The method is always the same: write the slow DP precisely, then study the shape of its inner loop. This lecture works through that method on real problems. Heavier optimisations (convex hull trick, divide and conquer, Knuth) belong to the National track (Lecture `nationals-03`).

2. Prefix-sum optimisation

2.1 The pattern

Whenever a transition reads a **contiguous range** of the previous layer and combines it with `+`,

```
dp[i][j] = Σ_{p = L(j)}^{R(j)} dp[i−1][p],
```

precompute prefix sums of layer `i − 1` and evaluate each state in O(1).

2.2 Example: AtCoder EDPC M — Candies

Distribute `K` candies among `N` children so that child `i` gets between 0 and `a_i`; count the ways modulo $10^9 + 7$ ($N \le 100$, $K \le 10^5$). The slow DP is `dp[i][j] = Σ_{x=0}^{a_i} dp[i−1][j − x]`, which is $O(N \cdot K^2) \approx 10^{12}$. The range of `j − x` is `[max(0, j − a_i), j]`, contiguous, so with `pre[j + 1] = dp[0] + … + dp[j]` each state is `pre[j + 1] − pre[lo]`. Total $O(N \cdot K) = 10^7$.

When subtracting modulo `p`, add `p` before taking `%`; in C++ `%` of a negative number is negative.

3. Monotonic deque optimisation

3.1 The pattern

When a transition takes a **max or min over a window** that slides monotonically as the state index increases,

```
dp[i][x] = f(x) + max_{x − w ≤ y ≤ x + w} dp[i−1][y],
```

a monotonic deque gives the window extremum in amortised O(1). The deque stores indices whose values are decreasing (for max). To add index `r`, pop from the back while the back's value is `≤ dp[r]` (it can never be the maximum again, since `r` is newer and at least as large). To move the left edge, pop from the front while the front index is outside the window.

3.2 Example: Codeforces 372C — Watching Fireworks is Fun

A street has sections 1..n (n ≤ 150 000). Firework `i` is launched at time `t_i` at section `a_i`; if you are at `x` you gain `b_i − |a_i − x|` (can be negative). You can move at most `d` sections per time unit and start anywhere. Maximise total happiness (m ≤ 300 fireworks).

Between fireworks `i − 1` and `i` you can move at most `w = d·(t_i − t_{i−1})` sections, so

```
dp_i[x] = b_i − |a_i − x| + max_{|y − x| ≤ w} dp_{i−1}[y].
```

The naive inner max is O(w) per state; with the deque it is O(1) amortised, for O(m·n) total. Two details: `w` can reach about $1.5 \cdot 10^{14}$, so compute it in `long long` and cap it at `n`; and as `x` moves right, both window edges move right, so the window really is sliding.

3.3 Variants

- *"Every window of length k must contain a chosen element"* — Codeforces 1077F2 *Pictures with Kittens (hard)*: choose exactly `x` of `n` pictures maximising beauty so that every segment of `k` consecutive pictures has at least one chosen. With `dp[j][i]` = best total when `j` pictures are chosen and the last is `i`, the predecessor `p` must satisfy `i − k ≤ p < i`: a sliding window per layer `j`.
- *"Jump at most k back"*: `dp[i] = c_i + min_{i−k ≤ p < i} dp[p]`.
- *Maximum sum of a subarray of length between a and b* (CSES 1644): with prefix sums `P`, the answer for right end `r` is `P[r] − min_{r−b ≤ l ≤ r−a} P[l]`, again a sliding window.

- If the allowed predecessors are not a window but satisfy "key < x", use a Fenwick or segment tree indexed by (compressed) keys instead (Lecture 6).

4. DP on subsequences

4.1 Occurrences of a pattern

How many times does `t` (length m) occur as a subsequence of `s`? Let `dp[j]` be the number of ways to match `t[0..j)` using the processed prefix of `s`. For each character `c` of `s`, for `j` from `m` down to 1, if `t[j−1] = c` then `dp[j] += dp[j−1]`. Iterating downwards ensures each character of `s` is used at most once per subsequence, exactly as in 0/1 knapsack.

4.2 Distinct subsequences

Let `ways` be the number of distinct subsequences (including the empty one) of the processed prefix. Appending character `c`, each old subsequence can be extended by `c` or not, giving `2·ways` candidates. The duplicates are precisely the subsequences ending in `c` that were already created when `c` last appeared; their number equals the value of `ways` just *before* that previous occurrence. So keep `last[c]`:

```
new = 2·ways − last[c];  last[c] = ways;  ways = new.
```

The answer for non-empty subsequences is `ways − 1`. Codeforces 1183H extends this to counting distinct subsequences by length.

4.3 Wildcards: Codeforces 1426F — Number of Subsequences

A string over `a, b, c, ?` with `q` question marks; over all `3^q` fillings, find the total number of subsequences equal to `abc`. Track totals across all fillings simultaneously:

- `cnt`: number of strings so far (`3^{#?}`),
- `a`, `ab`, `abc`: total occurrences of these subsequences summed over all strings.

Reading `a` adds one new `a`-subsequence to each string: `a += cnt`. Reading `b` extends every `a`: `ab += a`. Reading `c`: `abc += ab`. Reading `?` triples the set of strings; in one third the character is `a`, and so on:

```
a' = 3a + cnt,  ab' = 3ab + a,  abc' = 3abc + ab,  cnt' = 3cnt.
```

Compute all new values from the old ones before assigning. This "maintain counts for every prefix of the pattern" view generalises to any short pattern and is a favourite regional medium.

5. Reconstruction

Regional problems often say "print an optimal solution; if there are several, print the lexicographically smallest". Three methods:

1. **Parent pointers.** Store the argmax for each state and follow it back from the final state. Necessary when the DP was optimised (deque, LIS) and the table no longer contains enough information to re-trace.
2. **Re-tracing.** From the final state, find any predecessor whose value plus the transition cost equals the current value. No extra memory; standard for LCS and knapsack.
3. **Forward walk over a suffix DP.** Compute `best[state]` for the *suffix* starting at each state, then build the answer from the front, at each step choosing the smallest symbol that keeps the total optimal. This gives the lexicographically smallest optimal answer naturally, because every choice is made knowing the optimum of the rest.

5.1 LCS re-tracing (AtCoder EDPC F)

```
string res;
int i = n, j = m;
while (i > 0 && j > 0) {
    if (s[i - 1] == t[j - 1]) { res += s[i - 1]; i--; j--; }
    else if (dp[i - 1][j] >= dp[i][j - 1]) i--;
    else j--;
}
reverse(res.begin(), res.end());
```

5.2 LIS with parent pointers

The O(n log n) LIS keeps `tailVal[k]`, the smallest tail of an increasing subsequence of length `k + 1`, and `tailIdx[k]`, its index. When `a[i]` lands at position `k`, its parent is `tailIdx[k − 1]` at that moment. Following parents from `tailIdx.back()` yields an actual LIS. The array `tailVal` itself is generally **not** a valid subsequence. Use `lower_bound` for strictly increasing and `upper_bound` for non-decreasing subsequences.

6. Reference implementations

6.1 AtCoder EDPC M — Candies

```
#include <bits/stdc++.h>
using namespace std;
const int MOD = 1'000'000'007;

int main() {
```

```
    int n, k;
    cin >> n >> k;
    vector<long long> dp(k + 1, 0), pre(k + 2);
    dp[0] = 1;
    for (int i = 0; i < n; i++) {
        int a;
        cin >> a;
        pre[0] = 0;
        for (int j = 0; j <= k; j++) pre[j + 1] = (pre[j] + dp[j]) % MOD;
        for (int j = 0; j <= k; j++) {
            int lo = max(0, j - a);
            dp[j] = (pre[j + 1] - pre[lo] + MOD) % MOD;    // dp[lo..j]
        }
    }
    cout << dp[k] << '\n';
}
```

All four official samples pass (5, 0, 1, 665683269).

6.2 Codeforces 372C — Watching Fireworks is Fun

```
#include <bits/stdc++.h>
using namespace std;
using ll = long long;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, m;
    ll d;
    cin >> n >> m >> d;
    vector<ll> dp(n + 1, 0), nd(n + 1);
    ll prevT = 1;
    for (int i = 0; i < m; i++) {
        ll a, b, t;
        cin >> a >> b >> t;
        ll w = min<ll>(n, (t - prevT) * d);
        prevT = t;
        deque<int> dq;                                  // indices, values decreasing
        int right = 1;
        for (int x = 1; x <= n; x++) {
            while (right <= n && right <= x + w) {
                while (!dq.empty() && dp[dq.back()] <= dp[right]) dq.pop_back();
                dq.push_back(right++);
            }
            while (dq.front() < x - w) dq.pop_front();
            nd[x] = dp[dq.front()] + b - llabs(a - x);
        }
        swap(dp, nd);
    }
    cout << *max_element(dp.begin() + 1, dp.end()) << '\n';
}
```

Both official samples pass (−31 and 1992).

6.3 Number of distinct non-empty subsequences

```cpp
#include <bits/stdc++.h>
using namespace std;
const long long MOD = 1'000'000'007;

int main() {
    string s;
    cin >> s;
    long long ways = 1;                         // includes the empty subsequence
    array<long long, 26> last{};
    for (char ch : s) {
        int c = ch - 'a';
        long long nw = (2 * ways - last[c] + MOD) % MOD;
        last[c] = ways;
        ways = nw;
    }
    cout << (ways - 1 + MOD) % MOD << '\n';
}
```

Verified against brute-force enumeration on random strings of length ≤ 12.

6.4 Codeforces 1426F — Number of Subsequences

```cpp
#include <bits/stdc++.h>
using namespace std;
const long long MOD = 1'000'000'007;

int main() {
    int n;
    string s;
    cin >> n >> s;
    long long cnt = 1, a = 0, ab = 0, abc = 0;
    for (char ch : s) {
        if (ch == 'a') a = (a + cnt) % MOD;
        else if (ch == 'b') ab = (ab + a) % MOD;
        else if (ch == 'c') abc = (abc + ab) % MOD;
        else {
            long long na = (3 * a + cnt) % MOD;
            long long nab = (3 * ab + a) % MOD;
            long long nabc = (3 * abc + ab) % MOD;
            cnt = 3 * cnt % MOD;
            a = na, ab = nab, abc = nabc;
        }
    }
    cout << abc << '\n';
}
```

Official samples: `ac?b?c` → 24, `???????` → 2835, `cccbbbaaa` → 0, `a???c` → 46.

6.5 LIS with reconstruction

```cpp
#include <bits/stdc++.h>
using namespace std;

int main() {
    int n;
    cin >> n;
    vector<long long> a(n);
    for (auto& x : a) cin >> x;
    vector<int> tailIdx, parent(n, -1);
    vector<long long> tailVal;
    for (int i = 0; i < n; i++) {
        int k = lower_bound(tailVal.begin(), tailVal.end(), a[i]) - tailVal.begin();
        if (k == (int)tailVal.size()) { tailVal.push_back(a[i]); tailIdx.push_back(i); }
        else { tailVal[k] = a[i]; tailIdx[k] = i; }
        parent[i] = k > 0 ? tailIdx[k - 1] : -1;
    }
    vector<long long> seq;
    for (int i = tailIdx.empty() ? -1 : tailIdx.back(); i != -1; i = parent[i])
        seq.push_back(a[i]);
    reverse(seq.begin(), seq.end());
    cout << seq.size() << '\n';
    for (size_t i = 0; i < seq.size(); i++) cout << seq[i] << (i + 1 == seq.size() ? '\n'
: ' ');
}
```

Verified against an O(n²) DP: the printed sequence is strictly increasing, is a subsequence of the input, and has optimal length. On `3 1 4 1 5 9 2 6` it prints length 4 and `1 4 5 6`.

7. Common mistakes

- **Negative remainders** after modular subtraction.
- **Off-by-one window bounds** in the deque (`<` vs `<=`); test with `w = 0`.
- **Overflow of the window width** (`Δt · d`); cap it.
- **Updating state in place** when the new values depend on old ones (the `?` case in 1426F, distinct subsequences' `last[c]`).
- **Wrong iteration direction** in subsequence counting (forwards would reuse a character).
- **Reading `tailVal` as an LIS.**
- **Re-tracing an optimised DP** that no longer stores the information; store parents instead.

8. Practice problems

- AtCoder EDPC M — Candies: https://atcoder.jp/contests/dp/tasks/dp_m

- AtCoder EDPC F — LCS: https://atcoder.jp/contests/dp/tasks/dp_f
- Codeforces 372C — Watching Fireworks is Fun: https://codeforces.com/problemset/problem/372/C
- Codeforces 1077F2 — Pictures with Kittens (hard version): https://codeforces.com/problemset/problem/1077/F2
- Codeforces 1426F — Number of Subsequences: https://codeforces.com/problemset/problem/1426/F
- Codeforces 1183H — Subsequences (hard version): https://codeforces.com/problemset/problem/1183/H
- CSES 1145 — Increasing Subsequence: https://cses.fi/problemset/task/1145
- CSES 1093 — Two Sets II: https://cses.fi/problemset/task/1093
- CSES 1644 — Maximum Subarray Sum II: https://cses.fi/problemset/task/1644

9. Further reading

- AtCoder Educational DP Contest (https://atcoder.jp/contests/dp) — all 26 problems; at regional level you should be able to solve A–T comfortably.
- cp-algorithms: "Longest increasing subsequence" (https://cp-algorithms.com/sequences/longest_increasing_subsequence.html) and "Minimum stack / Minimum queue".
- Antti Laaksonen, *Competitive Programmer's Handbook*, Chapter 7 (dynamic programming).
- National track Lecture 3 (DP optimisations) continues where this lecture stops.